Building a Governed Chatbot with genesis_sdlc

GTL version: 0.3.0 **Engine**: abiogenesis 1.0.0 / genesis_sdlc 0.1.3

Contents

1. What We Are Building
2. Project Bootstrap
3. Intent
4. Requirements
5. Feature Decomposition
6. Design
7. Implementation Walkthrough
8. The Bot at Runtime — GTL as Dispatch Engine
9. Running the Build
10. Current Limitations and V2 Path

- Appendix A: Complete GTL Spec

---

1. What We Are Building

Nexus is a context-aware chatbot built using genesis_sdlc. It connects to a messaging platform (Slack, Teams, or any service with an adapter) and handles user commands and freeform requests.

The distinguishing characteristic: Nexus uses GTL's F_D → F_P escalation chain at runtime, not just during build. An incoming message is treated as an intent asset. The bot resolves it through deterministic checks first, escalates to an LLM only when deterministic resolution fails, and gates sensitive operations through explicit human approval. The same formal model that governed construction governs execution.

What Nexus does

Capability	Description
Command dispatch	Registered commands matched by pattern and executed deterministically
Knowledge resolution	Freeform queries resolved through a vector cache then LLM escalation
URI scanning	Recursive LLM traversal of a provided URI list; facts indexed into the vector cache
Pinned knowledge	Manually registered entries in the vector cache; returned at F_D layer with no LLM call
Sensitive gates	Operations flagged as requiring human approval pause and request confirmation

Capability	Description
Platform adapters	Slack, Teams, or any service implementing the `MessageAdapter` interface

How the runtime uses GTL

At build time: genesis_sdlc drives construction through the SDLC graph. Each edge (intent → requirements, design → code, etc.) is governed by GTL evaluators.

At runtime: Nexus runs its own micro-iterate loop on every incoming message. Each message is an intent. The resolution graph is:

```
chat_intent → resolved_response
```

With the following evaluator chain:

1. `F_D` — pattern-match registered commands
2. `F_D` — vector cache lookup (pinned data and cached F_P responses)
3. `F_P` — LLM resolution with assembled context (cache misses + relevant URI content)
4. `F_H` — human approval gate for sensitive operations

This is not metaphorical. The bot imports `gtl.core`, defines a resolution `Package`, and runs `iterate()` on each message.

---

2. Project Bootstrap

Prerequisites

- Python 3.11+
- genesis_sdlc installed: `pip install -e /path/to/genesis_sdlc`
- abiogenesis installed: `pip install -e /path/to/abiogenesis`
- Slack API token (or Teams/ICQ equivalent) for runtime; not needed for the build phase

Install into the project

```python
python -m genesis_sdlc.install \
    --target /path/to/nexus \
    --project-slug nexus \
    --platform python
```

After install, the project layout is:

```
nexus/
├── gtl_spec/
│   ├── GENESIS_BOOTLOADER.md         ← LLM constraint context (do not edit)
│   └── packages/
│       └── nexus.py                  ← Your spec (edit this)
├── builds/python/
│   ├── src/nexus/                    ← Implementation source
│   ├── tests/                        ← Test suite
│   └── design/adrs/                  ← Architecture decision records
├── .genesis/                         ← abiogenesis engine (do not edit)
├── .ai-workspace/
│   ├── events/events.jsonl           ← Append-only event log
│   └── features/active/              ← Feature vectors in progress
└── CLAUDE.md                         ← Project orientation + Bootloader
```

Verify the engine

```python
cd /path/to/nexus
PYTHONPATH=.genesis python -m genesis gaps --workspace .
```

The output shows `total_delta` across all edges. At this point all edges are unconverged — no work has been done yet.

Check current state

```
gen gaps --workspace .
```

```
{
  "total_delta": 6,
  "converged": false,
  "gaps": [
    { "edge": "intent→requirements", "delta": 1, "failing":
        ["intent_approved"] },
    { "edge": "requirements→feature_decomp", "delta": 3, "failing":
        ["req_coverage", "decomp_complete", "decomp_approved"] },
    ...
  ]
}
```

Six unconverged edges. The engine selects `intent→requirements` first — every graph traversal starts at the upstream end.

---

3. Intent

The first edge is `intent→requirements`. It has one evaluator: `intent_approved` (F_H). A human writes the intent document and approves it.

Create `INTENT.md`:

```
# Intent: Nexus

**ID**: INT-001
**Status**: Approved

## Problem

Teams communicate in Slack. Questions that can be answered from existing
        documentation,
system status endpoints, or simple lookups are routed to humans who have to
        find the
answer manually. This adds latency and pulls engineers away from focused
        work.

There is no single resolution path: some questions have deterministic answers
        (system
status, known commands), others require reading documentation (policies,
        runbooks),
others require judgment. A bot that handles only the deterministic case
        quickly becomes
unreliable because queries it cannot match fall through silently.

## Value Proposition

Nexus handles the full escalation chain:
- Registered commands execute deterministically with no LLM call
- Questions with cached answers return instantly from the vector cache
- Unknown questions are resolved by an LLM with relevant context assembled
        from the
  vector cache and configured URI sources
- Sensitive operations pause and request human approval before executing

The same GTL escalation model that governs the Nexus build governs Nexus at
        runtime.

## Scope (V1)

- Slack adapter (primary); Teams and other platforms via the MessageAdapter
        interface
- Command registration via YAML config
- Vector cache (ChromaDB) for pinned data and F_P response caching
- URI scanner: recursive LLM traversal with configurable depth
```

```
- Sensitive command gate: human approval via Slack reaction or DM
        confirmation
- Single-tenant deployment (one bot per workspace)
```

Emit the approval:

```
PYTHONPATH=.genesis python -m genesis emit-event \
    --type review_approved \
    --data '{"edge": "intent→requirements", "actor": "human"}'
```

Run `gen gaps` again:

```
{ "edge": "intent→requirements", "delta": 0, "passing": ["intent_approved"] }
```

The first edge is converged. The engine now selects `requirements→feature_decomp`.

---

4. Requirements

The `requirements→feature_decomp` edge runs an F_P agent assessment (`decomp_complete`) gated by an F_D coverage check (`req_coverage`) and a final F_H approval (`decomp_approved`).

Run one iteration to let the agent draft requirements:

```
gen start --workspace . --auto
```

The engine dispatches to the F_P agent with the intent document as context. The agent writes requirements in plain Markdown under `builds/python/design/requirements.md`.

The requirements for Nexus:

REQ Key	Requirement
`REQ-F-ADAPT-001`	The bot connects to a messaging platform via a MessageAdapter interface
`REQ-F-ADAPT-002`	Slack, Teams, and a test (local echo) adapter ship in V1
`REQ-F-DISPATCH-001`	Incoming messages resolve through an F_D → F_P → F_H evaluation chain
`REQ-F-DISPATCH-002`	F_D: registered commands are matched by pattern and executed without LLM
`REQ-F-DISPATCH-003`	F_D: the vector cache is checked before any LLM call
`REQ-F-DISPATCH-004`	F_P: LLM resolution fires only when F_D evaluators find no answer
`REQ-F-DISPATCH-005`	F_H: operations flagged `sensitive=true` require explicit human approval
`REQ-F-CACHE-001`	The vector cache stores pinned entries (manually registered)
`REQ-F-CACHE-002`	F_P responses with confidence ≥ 0.85 are written back to the cache
`REQ-F-CACHE-003`	Cache lookups return entries above a configurable similarity threshold
`REQ-F-URI-001`	The bot accepts a list of URIs via config and ingests them into the vector cache
`REQ-F-URI-002`	Ingestion is recursive: the LLM follows linked URIs up to a configurable depth

REQ Key	Requirement
REQ-F-URI-003	Ingested content is chunked and stored with source URI metadata
REQ-F-CFG-001	Platform credentials, cache backend, and URI list are configured via YAML
REQ-F-CFG-002	Command registry is declared in YAML; commands are not hardcoded
REQ-NFR-001	F_D responses return in under 100ms (no network calls)
REQ-NFR-002	F_P responses return in under 10s (LLM call with assembled context)

These requirements are written into `builds/python/design/requirements.md` and the REQ keys are registered in `Package.requirements` in `gtl_spec/packages/nexus.py`.

The F_D coverage check (`req_coverage`) passes when every key in `Package.requirements` appears in at least one feature vector. Feature vectors are written at the next edge.

Approve the requirements:

```
PYTHONPATH=.genesis python -m genesis emit-event \
    --type review_approved \
    --data '{"edge": "requirements→feature_decomp", "actor": "human"}'
```

---

5. Feature Decomposition

The engine writes feature vectors to `.ai-workspace/features/active/`. Each vector is a YAML file declaring which REQ keys it satisfies.

Five features cover the requirements:

`REQ-F-NEXUS-ADAPTER.yml` — messaging platform integration

```yaml
id: REQ-F-NEXUS-ADAPTER
title: Platform adapter and message ingestion
status: active
satisfies:
  - REQ-F-ADAPT-001
  - REQ-F-ADAPT-002
```

`REQ-F-NEXUS-DISPATCH.yml` — the runtime escalation chain

```yaml
id: REQ-F-NEXUS-DISPATCH
title: GTL-powered message dispatch (F_D → F_P → F_H)
status: active
satisfies:
  - REQ-F-DISPATCH-001
  - REQ-F-DISPATCH-002
  - REQ-F-DISPATCH-003
  - REQ-F-DISPATCH-004
  - REQ-F-DISPATCH-005
```

`REQ-F-NEXUS-CACHE.yml` — vector cache layer

```yaml
id: REQ-F-NEXUS-CACHE
title: Vector cache (ChromaDB) — pinned data and F_P response caching
status: active
satisfies:
  - REQ-F-CACHE-001
  - REQ-F-CACHE-002
  - REQ-F-CACHE-003
```

`REQ-F-NEXUS-URI.yml` — URI ingestion pipeline

```yaml
id: REQ-F-NEXUS-URI
title: Recursive URI scanner and knowledge ingestion
status: active
satisfies:
  - REQ-F-URI-001
  - REQ-F-URI-002
  - REQ-F-URI-003
```

`REQ-F-NEXUS-CFG.yml` — configuration and command registry

```yaml
id: REQ-F-NEXUS-CFG
title: YAML configuration and command registry
status: active
satisfies:
  - REQ-F-CFG-001
  - REQ-F-CFG-002
  - REQ-NFR-001
  - REQ-NFR-002
```

After writing these files, `req_coverage` passes:

```bash
gen check-req-coverage \
    --package gtl_spec.packages.nexus:package \
    --features .ai-workspace/features/
```

```
{ "covered": 17, "total": 17, "coverage": 1.0, "gaps": [] }
```

Approve the decomposition:

```bash
PYTHONPATH=.genesis python -m genesis emit-event \
    --type review_approved \
    --data '{"edge": "requirements→feature_decomp", "actor": "human"}'
```

---

6. Design

The `feature_decomp→design` edge produces architecture decision records. The agent drafts ADRs; a human approves the design before any code is written.

Five ADRs cover the design:

ADR-001: MessageAdapter interface

The adapter interface decouples platform-specific I/O from the dispatch logic.

```
MessageAdapter
  ├── receive() → ChatIntent
  ├── send(response: ChatResponse) → void
  └── request_approval(op: PendingOp) → ApprovalRequest

ChatIntent
  ├── id: str                    ← platform-specific message ID
  ├── text: str                  ← raw message text
  ├── user_id: str
  ├── channel_id: str
  ├── platform: str              ← "slack" | "teams" | "echo"
  └── metadata: dict
```

Adapter implementations: - `SlackAdapter` — uses the Slack Bolt SDK; connects via app token - `TeamsAdapter` — uses the Bot Framework SDK; webhook-based - `EchoAdapter` — local stdout/stdin; used in tests and development

ADR-002: Runtime dispatch as a GTL Package

The Nexus runtime defines its own GTL Package for message resolution. The resolution graph has two assets and one edge:

```
ChatIntent → ChatResponse
```

With five evaluators on the `resolve` edge:

Evaluator	Kind	Passes when
`command_match`	F_D	Message text matches a registered command pattern
`cache_hit`	F_D	Vector cache returns an entry above the similarity threshold
`pinned_hit`	F_D	A pinned entry matches the message topic
`llm_resolved`	F_P	LLM returns a response with confidence ≥ 0.7
`sensitive_approved`	F_H	Human has approved the pending operation

Evaluators run in the order listed. The first passing evaluator wins. The resolution loop stops when any single evaluator passes — this is a selector pattern, not a convergence pattern. The micro-iterate loop terminates on the first `delta = 0` rather than requiring all evaluators to pass.

This is a deliberate departure from the build-time semantics (where all evaluators must pass). The resolution `Package` uses `confirm="question"` on the edge — the edge converges when the question is answered by any evaluator.

ADR-003: Vector cache as a resolution layer

The vector cache (ChromaDB) serves three distinct data classes:

Class	Written by	Read at	Eviction
Pinned	Human operator via CLI	F_D `pinned_hit` evaluator	Never (until explicitly removed)
F_P response cache	Resolution loop after F_P pass	F_D `cache_hit` evaluator	TTL (configurable, default 7 days)
URI content	URI scanner pipeline	F_P context assembly	TTL (configurable, default 30 days)

The cache is not a knowledge base. It is a latency optimisation. Any entry can be evicted without losing correctness — the F_P evaluator will regenerate the answer if the cache misses. Pinned entries are the exception: they represent operator-validated facts that should always return at F_D speed.

Similarity threshold is configured per-deployment. The default is 0.82 for `cache_hit` and 0.95 for `pinned_hit` (pinned answers must be a very close match to avoid returning a wrong pinned fact for a slightly different question).

ADR-004: URI scanner pipeline

The URI scanner is a separate pipeline, not part of the message resolution loop. It runs on startup (if URIs have changed since last scan) and on demand via the `/nexus scan <url>` command.

```
uri_list → indexed_chunks
```

The scan pipeline:

1. **Fetch** — HTTP GET each URI; extract text (HTML → text, PDF → text, plain text as-is)

2. **Chunk** — split into ~512-token chunks with 64-token overlap
3. **Extract** — LLM reads each chunk and extracts: key facts, topic tags, linked URIs found in content
4. **Index** — write chunks to vector cache with metadata: `{source_uri, depth, topic_tags, extracted_at}`
5. **Recurse** — for each linked URI found, if `depth < max_depth`, add to scan queue

`max_depth` defaults to 1 (follow links one level deep from seed URIs). At depth 0 only the seed URIs are scanned.

The LLM extraction step prevents the cache from filling with irrelevant boilerplate (navbars, cookie notices, footer content). The LLM receives: the chunk text, the source URI, and the instruction "Extract the key factual content from this section. Discard navigation, advertisements, and boilerplate. Return nothing if there is no substantive content."

ADR-005: F_H gate for sensitive operations

Commands marked `sensitive: true` in the command registry pause the resolution loop and request human approval through the messaging platform itself.

Approval flow:

1. Nexus identifies a sensitive command match (F_D passes on `command_match`)
2. Nexus posts a confirmation message: "Requested: `deploy staging` — approve with ✅ or deny with ❌"
3. The message ID and pending operation are stored in the `PendingOps` table
4. The `sensitive_approved` F_H evaluator checks the `PendingOps` table for a resolution
5. The platform adapter listens for reactions; on ✅ or ❌, updates the `PendingOps` table
6. The resolution loop resumes on the next poll cycle

For Slack this uses message reactions. For Teams this uses an Adaptive Card with Approve/Deny buttons. The adapter interface abstracts the mechanism.

---

7. Implementation Walkthrough

The `design→code` and `code↔unit_tests` edges produce the implementation. This walkthrough shows the key modules and their structure. It is illustrative — not a complete implementation.

Module layout

```
builds/python/src/nexus/
├── __init__.py
├── adapter/
│   ├── base.py          ← MessageAdapter ABC
│   ├── slack.py         ← SlackAdapter
│   ├── teams.py         ← TeamsAdapter
│   └── echo.py          ← EchoAdapter (test/dev)
├── dispatch/
│   ├── package.py       ← Runtime GTL Package definition
│   ├── evaluators.py    ← F_D evaluators (command_match, cache_hit,
pinned_hit)
│   ├── loop.py          ← micro-iterate loop
│   └── context.py       ← Context assembly for F_P
├── cache/
│   ├── store.py         ← ChromaDB wrapper
│   ├── pinned.py        ← Pinned entry management
│   └── ttl.py           ← Eviction logic
├── scanner/
│   ├── pipeline.py      ← URI scan pipeline
│   ├── fetcher.py       ← HTTP fetch + text extraction
│   └── chunker.py       ← Text chunking with overlap
└── config.py            ← YAML config loader
```

The MessageAdapter interface

```python
# builds/python/src/nexus/adapter/base.py
# Implements: REQ-F-ADAPT-001

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional


@dataclass
class ChatIntent:
    id: str
    text: str
    user_id: str
    channel_id: str
    platform: str
    metadata: dict


@dataclass
class ChatResponse:
    text: str
    intent_id: str
    resolved_by: str        # "command" | "cache" | "pinned" | "llm" |
        "pending_approval"
    confidence: float
    sources: list[str]      # URIs or cache entry IDs that contributed


class MessageAdapter(ABC):
    """Platform-agnostic messaging interface."""

    @abstractmethod
    def receive(self) -> ChatIntent:
        """Block until the next message arrives. Returns a ChatIntent."""

    @abstractmethod
    def send(self, response: ChatResponse) -> None:
        """Send a response back to the originating channel."""

    @abstractmethod
    def request_approval(self, intent: ChatIntent, description: str) -> str:
        """
        Post a confirmation request and return a pending_op_id.
        The approval or denial is recorded asynchronously.
        """
```

The runtime dispatch Package

```python
# builds/python/src/nexus/dispatch/package.py
# Implements: REQ-F-DISPATCH-001

from gtl.core import (
    Asset, Edge, Evaluator, Job, Operator, Package, Worker,
    F_D, F_P, F_H,
)

# Assets
chat_intent  = Asset(name="chat_intent",  id_format="MSG-{SEQ}",
                     markov=["text_present", "user_identified"])
chat_response = Asset(name="chat_response", id_format="RSP-{SEQ}",
                      lineage=[chat_intent],
                      markov=["text_present", "resolved_by_known"])

# Operators
fd_resolver = Operator("fd_resolver", F_D,
        "exec://nexus.dispatch.evaluators:run_fd")
fp_resolver = Operator("fp_resolver", F_P, "agent://claude/nexus")
fh_gate     = Operator("fh_gate",     F_H, "fh://pending_ops")

# Edge: resolve a chat_intent into a chat_response
resolve = Edge(
    name="resolve",
    source=chat_intent,
    target=chat_response,
    using=[fd_resolver, fp_resolver, fh_gate],
    confirm="question",   # first passing evaluator wins; not all-must-pass
```

```
)

# Evaluators
command_match = Evaluator(
    "command_match", F_D,
    "Message text matches a registered command pattern; response constructed
        deterministically",
    command="nexus.dispatch.evaluators:eval_command_match",
)
cache_hit = Evaluator(
    "cache_hit", F_D,
    "Vector cache returns an entry above the configured similarity
        threshold",
    command="nexus.dispatch.evaluators:eval_cache_hit",
)
pinned_hit = Evaluator(
    "pinned_hit", F_D,
    "A pinned entry matches the message topic above the pinned threshold
        (0.95)",
    command="nexus.dispatch.evaluators:eval_pinned_hit",
)
llm_resolved = Evaluator(
    "llm_resolved", F_P,
    "LLM assembles context from cache and URI content and returns a
        response",
)
sensitive_approved = Evaluator(
    "sensitive_approved", F_H,
    "A pending_op entry for this intent exists with status=approved",
)

# Job
resolve_job = Job(
    edge=resolve,
    evaluators=[pinned_hit, command_match, cache_hit, llm_resolved,
        sensitive_approved],
)

# Package and Worker
resolution_package = Package(
    name="nexus_resolution",
    assets=[chat_intent, chat_response],
    edges=[resolve],
    operators=[fd_resolver, fp_resolver, fh_gate],
)

resolution_worker = Worker(
    id="nexus_runtime",
    can_execute=[resolve_job],
)
```

The F_D evaluators

```
# builds/python/src/nexus/dispatch/evaluators.py
# Implements: REQ-F-DISPATCH-002
# Implements: REQ-F-DISPATCH-003

import re
from nexus.cache.store import VectorStore
from nexus.config import NexusConfig


def eval_command_match(intent, config: NexusConfig) -> tuple[bool, dict]:
    """
    Check registered commands for a pattern match.
    Returns (passed, result_dict).
    """
    text = intent.text.strip()
    for cmd in config.commands:
        if re.fullmatch(cmd.pattern, text, re.IGNORECASE):
            if cmd.sensitive:
                # Let the F_H evaluator handle this
                return False, {"reason": "sensitive_command", "command":
        cmd.name}
            result = cmd.execute(intent, config)
            return True, {"resolved_by": "command", "command": cmd.name,
                          "text": result, "confidence": 1.0, "sources": []}
    return False, {"reason": "no_command_match"}
```

```python
def eval_pinned_hit(intent, store: VectorStore, threshold: float = 0.95) ->
        tuple[bool, dict]:
    """
    Check the pinned partition of the vector cache.
    Pinned entries always return at F_D speed — no LLM call.
    """
    results = store.query(intent.text, partition="pinned", n_results=1)
    if results and results[0].score >= threshold:
        return True, {
            "resolved_by": "pinned",
            "text": results[0].text,
            "confidence": results[0].score,
            "sources": [results[0].source_uri],
        }
    return False, {"reason": "pinned_below_threshold", "best_score":
        results[0].score if results else 0.0}


def eval_cache_hit(intent, store: VectorStore, threshold: float = 0.82) ->
        tuple[bool, dict]:
    """
    Check the F_P response cache. Returns cached LLM responses for recurring
        queries.
    """
    results = store.query(intent.text, partition="fp_cache", n_results=1)
    if results and results[0].score >= threshold:
        return True, {
            "resolved_by": "cache",
            "text": results[0].text,
            "confidence": results[0].score,
            "sources": results[0].sources,
        }
    return False, {"reason": "cache_below_threshold", "best_score":
        results[0].score if results else 0.0}
```

The micro-iterate loop

```python
# builds/python/src/nexus/dispatch/loop.py
# Implements: REQ-F-DISPATCH-001
# Implements: REQ-F-DISPATCH-004
# Implements: REQ-F-DISPATCH-005

from nexus.dispatch.package import resolve_job
from nexus.dispatch.context import assemble_context
from nexus.cache.store import VectorStore


def resolve_intent(intent, config, store: VectorStore, llm_client,
        pending_ops) -> dict:
    """
    Run the micro-iterate loop on a single ChatIntent.
    Returns a result dict with keys: resolved_by, text, confidence, sources.
    """
    # Phase 1: F_D evaluators — no LLM call
    passed, result = eval_pinned_hit(intent, store, config.pinned_threshold)
    if passed:
        return result

    passed, result = eval_command_match(intent, config)
    if passed:
        return result
    if result.get("reason") == "sensitive_command":
        op_id = pending_ops.create(intent, result["command"])
        return {"resolved_by": "pending_approval", "text":
        _approval_message(result["command"]),
                "op_id": op_id, "confidence": 1.0, "sources": []}

    passed, result = eval_cache_hit(intent, store, config.cache_threshold)
    if passed:
        return result

    # Phase 2: F_P — assemble context and call LLM
    context = assemble_context(intent, store, config)
    response = llm_client.complete(
        system=context.system_prompt,
        user=intent.text,
```

```
        context_docs=context.docs,
    )

    if response.confidence >= 0.85:
        store.write(
            text=response.text,
            query=intent.text,
            partition="fp_cache",
            sources=response.sources,
            ttl_days=config.fp_cache_ttl_days,
        )

    return {
        "resolved_by": "llm",
        "text": response.text,
        "confidence": response.confidence,
        "sources": response.sources,
    }
```

The URI scanner

```
# builds/python/src/nexus/scanner/pipeline.py
# Implements: REQ-F-URI-001
# Implements: REQ-F-URI-002
# Implements: REQ-F-URI-003

from collections import deque
from nexus.scanner.fetcher import fetch_text
from nexus.scanner.chunker import chunk_text
from nexus.cache.store import VectorStore


def scan_uri_list(uris: list[str], store: VectorStore, llm_client, max_depth:
        int = 1) -> dict:
    """
    Scan a list of seed URIs and index extracted content into the vector
        cache.
    Recursively follows linked URIs up to max_depth.
    Returns a summary: {scanned, chunks_indexed, uris_followed, errors}.
    """
    queue = deque((uri, 0) for uri in uris)
    visited = set()
    stats = {"scanned": 0, "chunks_indexed": 0, "uris_followed": 0, "errors":
        []}

    while queue:
        uri, depth = queue.popleft()
        if uri in visited:
            continue
        visited.add(uri)

        try:
            text = fetch_text(uri)
        except Exception as e:
            stats["errors"].append({"uri": uri, "error": str(e)})
            continue

        chunks = chunk_text(text, chunk_size=512, overlap=64)

        for chunk in chunks:
            extraction = llm_client.complete(
                system=(
                    "Extract the key factual content from this section. "
                    "Discard navigation, advertisements, and boilerplate. "
                    "Return nothing if there is no substantive content. "
                    "Also list any linked URLs you find in the content."
                ),
                user=chunk,
            )

            if extraction.text.strip():
                store.write(
                    text=extraction.text,
                    partition="uri_content",
                    metadata={"source_uri": uri, "depth": depth,
        "topic_tags": extraction.tags},
                    ttl_days=30,
                )
```

```
                stats["chunks_indexed"] += 1

            if depth < max_depth:
                for linked_uri in extraction.linked_uris:
                    if linked_uri not in visited:
                        queue.append((linked_uri, depth + 1))
                        stats["uris_followed"] += 1

        stats["scanned"] += 1

    return stats
```

Context assembly for F_P

```
# builds/python/src/nexus/dispatch/context.py
# Implements: REQ-F-DISPATCH-004

from nexus.cache.store import VectorStore


def assemble_context(intent, store: VectorStore, config) ->
        "AssembledContext":
    """
    Assemble the context package sent to the LLM.
    Retrieves the most relevant chunks from all cache partitions.
    """
    # Pull from URI content cache — what the scan pipeline indexed
    uri_docs = store.query(intent.text, partition="uri_content", n_results=5)

    # Pull from pinned — even below the F_D threshold, pinned entries add
        signal
    pinned_docs = store.query(intent.text, partition="pinned", n_results=3)

    system_prompt = (
        "You are Nexus, a context-aware assistant embedded in this team's
        Slack workspace. "
        "Answer using only the provided context documents. "
        "If the answer cannot be determined from context, say so. "
        "Confidence: respond with a confidence score between 0 and 1 after
        your answer, "
        "on its own line as: CONFIDENCE: 0.XX"
    )

    return AssembledContext(
        system_prompt=system_prompt,
        docs=[d.text for d in pinned_docs + uri_docs],
        sources=[d.source_uri for d in pinned_docs + uri_docs],
    )
```

---

8. The Bot at Runtime — GTL as Dispatch Engine

This section describes what happens when a user sends a message to Nexus in Slack.

Message received

A user types: `@nexus what is the on-call rotation for the data team?`

The `SlackAdapter` converts this to:

```
ChatIntent(
    id="msg_abc123",
    text="what is the on-call rotation for the data team?",
    user_id="U012345",
    channel_id="C998877",
    platform="slack",
    metadata={"ts": "1742100000.123", "thread_ts": None},
)
```

Phase 1: F_D evaluation

`eval_pinned_hit` runs first. Suppose the ops team has pinned the on-call schedule as a known answer. The vector cache returns a match with score 0.97 (above the 0.95 threshold).

```
pinned_hit: PASS (score: 0.97)
resolved_by: pinned
text: "The data team on-call rotation is weekly, starting Monday..."
sources: ["pinned:data_team_oncall_v3"]
```

The loop terminates. No LLM call. Response time: < 100ms. The adapter sends the pinned answer back to the channel.

Phase 1: F_D evaluation — cache miss scenario

Next day, the same user asks: `@nexus how do I request access to the datalake?`

No pinned entry matches. `eval_command_match` finds no registered command. `eval_cache_hit` returns a match at score 0.79 — below the 0.82 threshold.

All F_D evaluators fail. The loop moves to Phase 2.

Phase 2: F_P escalation

`assemble_context` retrieves the five most relevant chunks from the URI content cache (the bot scanned the team's Confluence at startup). It builds a prompt:

```
System: You are Nexus... [system prompt]

Context document 1 (source: confluence.company.com/data-access):
  To request access to the datalake, file a ticket in the #data-access
channel...

Context document 2 (source: confluence.company.com/permissions):
  Data access requests are reviewed within 2 business days...

[3 more context documents]

User: how do I request access to the datalake?
```

The LLM returns a response with confidence 0.91. Since confidence ≥ 0.85, the response is written back to the F_P response cache. The next user who asks a similar question will hit `eval_cache_hit` at F_D speed.

Phase 3: F_H gate

A user types: `@nexus deploy staging`

`eval_command_match` matches the pattern `deploy (\w+)` with `sensitive: true`.

The loop returns immediately with `resolved_by: pending_approval` and posts:

```
Nexus: Requested: deploy staging — approve with ✅ or deny with ❌
       Requested by: @alice | Command: deploy staging
```

The resolution loop is suspended. The `PendingOps` table holds:

```
{
  "op_id": "op_xyz",
  "intent_id": "msg_def456",
  "command": "deploy_staging",
  "status": "pending",
  "requested_by": "U012345",
  "created_at": "..."
}
```

When any user with the `ops` role reacts with ✅, the `SlackAdapter` updates the entry to `status: approved`. The next poll cycle runs `sensitive_approved` which finds the entry and clears the gate. The deploy command executes.

The URI scan command

A user types: `@nexus scan https://docs.company.com/runbooks`

This matches the registered command `scan (.+)` (not sensitive). The command calls `scan_uri_list` with `max_depth=1`.

The scanner fetches the runbooks index, extracts 12 runbook links, and follows each one. Each runbook is chunked, LLM-extracted, and indexed into the `uri_content` partition of the vector cache. The scan completes in ~45 seconds.

```
Nexus: Scan complete — 13 URIs scanned, 147 chunks indexed, 0 errors
```

All future questions about runbook procedures will resolve from the cache without further web requests.

---

9. Running the Build

Full auto run

```
gen start --workspace . --auto --human-proxy
```

`--human-proxy` allows the engine to act as the F_H evaluator during unattended runs. It evaluates each gate criteria explicitly and writes a proxy log before emitting any approval event.

The engine traverses:

1. `intent→requirements` — writes `requirements.md`, awaits human approval
2. `requirements→feature_decomp` — runs F_D coverage check, dispatches F_P to write feature vectors, awaits human approval
3. `feature_decomp→design` — dispatches F_P to draft ADRs, awaits human approval
4. `design→code` — runs `impl_tags` F_D check, dispatches F_P to implement modules
5. `code↔unit_tests` — runs `tests_pass` and `validates_tags` F_D checks, dispatches F_P to fill test gaps

What the engine produces

After convergence, the project contains:

```
builds/python/
├── src/nexus/
│   ├── adapter/           ← SlackAdapter, TeamsAdapter, EchoAdapter
│   ├── dispatch/          ← GTL resolution package, evaluators, loop,
context
│   ├── cache/             ← ChromaDB store, pinned entries, TTL
│   ├── scanner/           ← URI pipeline, fetcher, chunker
│   └── config.py
└── tests/
    ├── test_adapter.py    ← Validates: REQ-F-ADAPT-001, REQ-F-ADAPT-002
    ├── test_dispatch.py   ← Validates: REQ-F-DISPATCH-001..005
    ├── test_cache.py      ← Validates: REQ-F-CACHE-001..003
    ├── test_scanner.py    ← Validates: REQ-F-URI-001..003
    └── test_config.py     ← Validates: REQ-F-CFG-001, REQ-F-CFG-002
```

Checking convergence

```
gen gaps --workspace .
```

```
{
  "total_delta": 0,
  "converged": true,
  "jobs_considered": 5,
  "gaps": [
    { "edge": "intent→requirements",       "delta": 0, "passing":
        ["intent_approved"] },
    { "edge": "requirements→feature_decomp","delta": 0, "passing":
        ["req_coverage", "decomp_complete", "decomp_approved"] },
```

```
    { "edge": "feature_decomp→design",     "delta": 0, "passing":
        ["design_coherent", "design_approved"] },
    { "edge": "design→code",               "delta": 0, "passing":
        ["impl_tags", "code_complete"] },
    { "edge": "code↔unit_tests",           "delta": 0, "passing":
        ["tests_pass", "validates_tags", "coverage_complete"] }
  ]
}
```

Running Nexus

After the build converges:

```
cd /path/to/nexus

# Configure credentials and URI list
cat > nexus.yml << 'EOF'
platform: slack
slack_app_token: xapp-...
slack_bot_token: xoxb-...
cache_dir: .nexus/cache
fp_cache_ttl_days: 7
uri_ttl_days: 30
cache_threshold: 0.82
pinned_threshold: 0.95
scan_depth: 1
uris:
  - https://docs.company.com/runbooks
  - https://confluence.company.com/data-access
commands:
  - name: help
    pattern: "help|\\?"
    response: "Available commands: help, status, scan <url>, deploy <env>"
  - name: status
    pattern: "status"
    exec: "python -m nexus.commands.status"
  - name: deploy
    pattern: "deploy (staging|production)"
    exec: "python -m nexus.commands.deploy {env}"
    sensitive: true
EOF

python -m nexus.main --config nexus.yml
```

---

10. Current Limitations and V2 Path

V1 limitations

The micro-iterate loop is synchronous per message. High-concurrency deployments (>10 simultaneous users) require running multiple instances behind a load balancer — V1 has no built-in concurrency model.

The F_P evaluator calls the LLM directly via the `llm_client` interface. In V1, this is a blocking call. Long LLM responses (>10s) block the adapter's receive loop. An async dispatch model is V2.

The `PendingOps` store is in-memory. Nexus restarts clear pending approvals. A persistent store (SQLite or Redis) is V2.

Multi-platform simultaneous deployment (Slack + Teams from one process) is not tested in V1. The `MessageAdapter` interface supports it structurally; the runtime does not.

The UAT edge

The `unit_tests→uat_tests` edge in the Nexus build spec requires a sandbox install with live Slack credentials. This edge is defined; the sandbox report evaluator checks for `.ai-workspace/uat/sandbox_report.json`. A staging Slack workspace with test credentials is needed to converge this edge.

V2 path

The natural V2 additions: - Async dispatch model (message queue + worker pool) - Persistent PendingOps store - Multi-platform simultaneous deployment - Per-user context: the bot learns which users ask which classes of question and pre-assembles personalised context - The URI scan pipeline feeding back into the bot's resolution Package via feature proposals — Nexus becomes a Genesis-enabled system that detects its own knowledge gaps and proposes new URI scans

---

Appendix A: Complete GTL Spec

This is the complete `gtl_spec/packages/nexus.py` for the Nexus project. Every REQ key from §4 is registered. The evaluator commands are wired to real subprocesses. This file governs the build.

```python
"""
nexus — GTL Package spec

This file IS the spec. The type system is the law.

  Asset.markov     → acceptance criteria for that asset type
  Job.evaluators   → convergence tests for each edge
  Edge.context     → constraint surface for each transition
  Worker           → who executes what

Nexus build graph:
    intent → requirements → feature_decomp → design → code ↔ unit_tests

F_D evaluators must be acyclic — never invoke genesis subcommands from
        pytest.
Run as:
    PYTHONPATH=.genesis python -m genesis <command> --workspace .
"""
# Implements: REQ-F-CFG-001

from gtl.core import (
    Package, Asset, Edge, Operator, Rule, Context, Evaluator, Job, Worker,
    F_D, F_P, F_H, consensus,
    OPERATIVE_ON_APPROVED, OPERATIVE_ON_APPROVED_NOT_SUPERSEDED,
)


# ── Contexts ──────────────────────────────────────────────────────────

bootloader = Context(
    name="bootloader",
    locator="workspace://gtl_spec/GENESIS_BOOTLOADER.md",
    digest="sha256:" + "0" * 64,
)

this_spec = Context(
    name="nexus_spec",
    locator="workspace://gtl_spec/packages/nexus.py",
    digest="sha256:" + "0" * 64,
)

intent_doc = Context(
    name="intent",
    locator="workspace://INTENT.md",
    digest="sha256:" + "0" * 64,
)

design_adrs = Context(
    name="design_adrs",
    locator="workspace://builds/python/design/adrs/",
    digest="sha256:" + "0" * 64,
)


# ── Operators ─────────────────────────────────────────────────────────

claude_agent  = Operator("claude_agent",  F_P, "agent://claude/genesis")
```

```
human_gate    = Operator("human_gate",    F_H, "fh://single")
pytest_op     = Operator("pytest",        F_D, "exec://python -m pytest
        builds/python/tests/ -q -m 'not e2e'")
check_impl_op = Operator("check_impl",    F_D, "exec://python -m genesis
        check-tags --type implements --path builds/python/src/")
check_test_op = Operator("check_test",    F_D, "exec://python -m genesis
        check-tags --type validates --path builds/python/tests/")


# ── Rules ──────────────────────────────────────────────────────────────

standard_gate = Rule(
    "standard_gate", approve=consensus(1, 1), dissent="recorded"
)


# ── Assets ─────────────────────────────────────────────────────────────

intent = Asset(
    name="intent",
    id_format="INT-{SEQ}",
    markov=["problem_stated", "value_proposition_clear", "scope_bounded"],
)

requirements = Asset(
    name="requirements",
    id_format="REQ-{SEQ}",
    lineage=[intent],
    markov=["keys_testable", "intent_covered", "no_implementation_details"],
    operative=OPERATIVE_ON_APPROVED,
)

feature_decomp = Asset(
    name="feature_decomp",
    id_format="FD-{SEQ}",
    lineage=[requirements],
    markov=["all_req_keys_covered", "dependency_dag_acyclic",
        "mvp_boundary_defined"],
    operative=OPERATIVE_ON_APPROVED,
)

design = Asset(
    name="design",
    id_format="DES-{SEQ}",
    lineage=[feature_decomp],
    markov=["adrs_recorded", "tech_stack_decided", "interfaces_specified",
            "runtime_gtl_package_defined", "no_implementation_details"],
    operative=OPERATIVE_ON_APPROVED_NOT_SUPERSEDED,
)

code = Asset(
    name="code",
    id_format="CODE-{SEQ}",
    lineage=[design],
    markov=["implements_tags_present", "importable",
        "adapter_interface_complete",
            "dispatch_loop_complete", "scanner_pipeline_complete",
        "no_v2_features"],
)

unit_tests = Asset(
    name="unit_tests",
    id_format="TEST-{SEQ}",
    lineage=[code],
    markov=["all_pass", "validates_tags_present",
        "all_req_keys_covered_by_tests"],
)

uat_tests = Asset(
    name="uat_tests",
    id_format="UAT-{SEQ}",
    lineage=[unit_tests],
    markov=["sandbox_install_passes", "e2e_slack_scenarios_pass",
        "accepted_by_human"],
)
```

```
# ── Edges ──────────────────────────────────────────────────────────────

e_intent_req = Edge(
    name="intent→requirements",
    source=intent,
    target=requirements,
    using=[claude_agent, human_gate],
    rule=standard_gate,
    context=[bootloader, this_spec],
)

e_req_feat = Edge(
    name="requirements→feature_decomp",
    source=requirements,
    target=feature_decomp,
    using=[claude_agent, human_gate],
    rule=standard_gate,
    context=[bootloader, this_spec, intent_doc],
)

e_feat_design = Edge(
    name="feature_decomp→design",
    source=feature_decomp,
    target=design,
    using=[claude_agent, human_gate],
    rule=standard_gate,
    context=[bootloader, this_spec, intent_doc],
)

e_design_code = Edge(
    name="design→code",
    source=design,
    target=code,
    using=[claude_agent, check_impl_op],
    context=[bootloader, this_spec, design_adrs],
)

e_tdd = Edge(
    name="code↔unit_tests",
    source=[code, unit_tests],
    target=unit_tests,
    co_evolve=True,
    using=[claude_agent, pytest_op, check_impl_op, check_test_op],
    context=[bootloader, this_spec, design_adrs],
)

e_unit_uat = Edge(
    name="unit_tests→uat_tests",
    source=unit_tests,
    target=uat_tests,
    using=[claude_agent, human_gate],
    rule=standard_gate,
    context=[bootloader, this_spec, design_adrs],
)


# ── Evaluators ─────────────────────────────────────────────────────────

# intent→requirements
eval_intent_fh = Evaluator(
    "intent_approved", F_H,
    "Human confirms: problem stated, value proposition clear, scope bounded,
        "
    "GTL runtime dispatch model described in intent",
)

# requirements→feature_decomp
eval_req_coverage = Evaluator(
    "req_coverage", F_D,
    "Every REQ key in Package.requirements appears in ≥1 feature vector
        satisfies: field",
    command="python -m genesis check-req-coverage --package
        gtl_spec.packages.nexus:package --features .ai-workspace/features/",
)
eval_decomp_fp = Evaluator(
    "decomp_complete", F_P,
```

```
    "Write feature vectors for all uncovered REQ keys to .ai-
        workspace/features/active/. "
    "Each vector must carry a satisfies: list. The runtime dispatch feature
        (REQ-F-DISPATCH-*) "
    "must be its own vector — it is architecturally central.",
)
eval_decomp_fh = Evaluator(
    "decomp_approved", F_H,
    "Human approves: feature set complete, dependency order correct, "
    "runtime dispatch feature decomposed separately from build-time
        features",
)

# feature_decomp→design
eval_design_fp = Evaluator(
    "design_coherent", F_P,
    "Agent: five ADRs present and coherent — MessageAdapter interface,
        runtime GTL Package, "
    "vector cache data classes, URI scanner pipeline, F_H sensitive gate. "
    "The runtime dispatch Package is defined with confirm='question'
        semantics. "
    "No implementation details in ADRs.",
)
eval_design_fh = Evaluator(
    "design_approved", F_H,
    "Human approves design before any code is written. "
    "Specifically confirms: runtime GTL Package is correctly modelled, "
    "vector cache partitions are clearly separated, URI scanner depth is
        configurable.",
)

# design→code
eval_impl_tags = Evaluator(
    "impl_tags", F_D,
    "All source files carry at least one # Implements: REQ-* tag",
    command="python -m genesis check-tags --type implements --path
        builds/python/src/",
)
eval_code_fp = Evaluator(
    "code_complete", F_P,
    "Agent: code implements all five features per ADRs. "
    "MessageAdapter ABC is present with receive/send/request_approval
        methods. "
    "Dispatch loop uses GTL resolution_package with confirm='question'
        semantics. "
    "F_D evaluators (command_match, cache_hit, pinned_hit) run before any LLM
        call. "
    "URI scanner follows links up to max_depth. "
    "No V2 features (async dispatch, persistent PendingOps, multi-platform
        simultaneous).",
)

# code↔unit_tests
eval_tests_pass = Evaluator(
    "tests_pass", F_D,
    "pytest: zero failures, zero errors (excluding e2e tests)",
    command="python -m pytest builds/python/tests/ -q --tb=short -m 'not
        e2e'",
)
eval_test_tags = Evaluator(
    "validates_tags", F_D,
    "All test files carry at least one # Validates: REQ-* tag",
    command="python -m genesis check-tags --type validates --path
        builds/python/tests/",
)
eval_coverage_fp = Evaluator(
    "coverage_complete", F_P,
    "Agent: every REQ-F-* key has at least one test that exercises the
        behaviour it describes. "
    "Specifically: REQ-F-DISPATCH-002 has tests for F_D-before-F_P ordering;
        "
    "REQ-F-CACHE-002 has a test that verifies cache write on high-confidence
        F_P response; "
    "REQ-F-URI-002 has a test that verifies recursive link following respects
        max_depth.",
)

# unit_tests→uat_tests
eval_uat_report = Evaluator(
```

```
    "uat_sandbox_report", F_D,
    "Sandbox e2e report exists at .ai-workspace/uat/sandbox_report.json with
        all_pass: true",
    command=(
        "python -c \""
        "import json,sys,pathlib; "
        "r=pathlib.Path('.ai-workspace/uat/sandbox_report.json'); "
        "d=json.loads(r.read_text()) if r.exists() else {}; "
        "sys.exit(0 if d.get('all_pass') and d.get('install_success') else
        1)"
        "\""
    ),
)
eval_uat_fp = Evaluator(
    "uat_e2e_passed", F_P,
    "Install into a fresh sandbox, connect to the staging Slack workspace, "
    "run the end-to-end scenarios: "
    "(1) pinned entry returns at F_D speed without LLM call; "
    "(2) unknown question escalates to LLM and caches the response; "
    "(3) sensitive command triggers approval gate and executes on approval; "
    "(4) /nexus scan indexes a test URI and the next question resolves from
        cache. "
    "Write results to .ai-workspace/uat/sandbox_report.json.",
)
eval_uat_fh = Evaluator(
    "uat_accepted", F_H,
    "Human confirms all four e2e scenarios passed in the staging workspace. "
    "No feature ships without live Slack proof.",
)


# ── Jobs ──────────────────────────────────────────────────────────────────

job_intent_req  = Job(e_intent_req,  [eval_intent_fh])
job_req_feat    = Job(e_req_feat,    [eval_req_coverage, eval_decomp_fp,
        eval_decomp_fh])
job_feat_design = Job(e_feat_design, [eval_design_fp, eval_design_fh])
job_design_code = Job(e_design_code, [eval_impl_tags, eval_code_fp])
job_tdd         = Job(e_tdd,         [eval_tests_pass, eval_test_tags,
        eval_coverage_fp])
job_uat         = Job(e_unit_uat,    [eval_uat_report, eval_uat_fp,
        eval_uat_fh])


# ── Worker ────────────────────────────────────────────────────────────────

worker = Worker(
    id="claude_code",
    can_execute=[job_intent_req, job_req_feat, job_feat_design,
        job_design_code, job_tdd, job_uat],
)


# ── Package ───────────────────────────────────────────────────────────────

package = Package(
    name="nexus",
    assets=[intent, requirements, feature_decomp, design, code, unit_tests,
        uat_tests],
    edges=[e_intent_req, e_req_feat, e_feat_design, e_design_code, e_tdd,
        e_unit_uat],
    operators=[claude_agent, human_gate, pytest_op, check_impl_op,
        check_test_op],
    rules=[standard_gate],
    contexts=[bootloader, this_spec, intent_doc, design_adrs],
    requirements=[
        # Adapter
        "REQ-F-ADAPT-001",   # MessageAdapter interface
        "REQ-F-ADAPT-002",   # Slack, Teams, Echo adapters in V1
        # Runtime dispatch
        "REQ-F-DISPATCH-001",  # F_D → F_P → F_H evaluation chain
        "REQ-F-DISPATCH-002",  # F_D: command pattern match
        "REQ-F-DISPATCH-003",  # F_D: vector cache check before LLM
        "REQ-F-DISPATCH-004",  # F_P: LLM fires only on F_D miss
        "REQ-F-DISPATCH-005",  # F_H: sensitive=true commands gate on
        approval
```

```
        # Vector cache
        "REQ-F-CACHE-001",   # Pinned entries (manually registered, never
        evicted)
        "REQ-F-CACHE-002",   # F_P responses cached on confidence ≥ 0.85
        "REQ-F-CACHE-003",   # Cache lookup above configurable similarity
        threshold
        # URI scanner
        "REQ-F-URI-001",     # URI list ingested into vector cache on startup
        "REQ-F-URI-002",     # Recursive link following up to configurable
        max_depth
        "REQ-F-URI-003",     # Chunks stored with source URI metadata
        # Configuration
        "REQ-F-CFG-001",     # YAML config: credentials, cache backend, URI
        list
        "REQ-F-CFG-002",     # Command registry in YAML (not hardcoded)
        # NFR
        "REQ-NFR-001",       # F_D responses < 100ms
        "REQ-NFR-002",       # F_P responses < 10s
    ],
)


if __name__ == "__main__":
    import json
    print(json.dumps({
        "package": package.name,
        "assets": [a.name for a in package.assets],
        "edges": [e.name for e in package.edges],
        "jobs": len(worker.can_execute),
        "worker": worker.id,
        "requirements": len(package.requirements),
    }, indent=2))
```